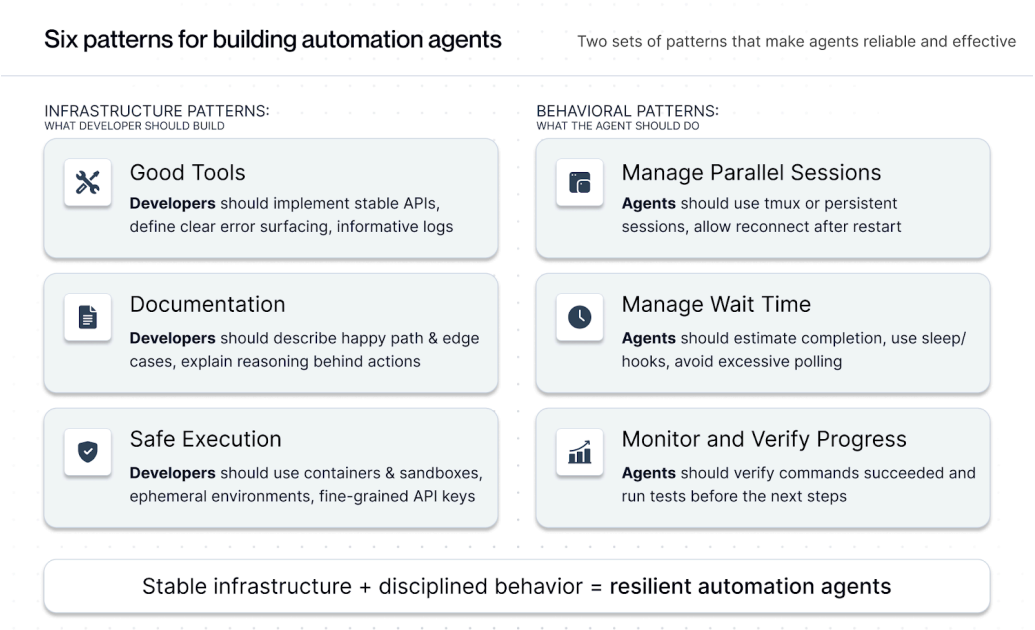
RESEARCH

How Together AI Uses AI Agents to Automate Complex Engineering Tasks: Lessons from Developing Efficient LLM Inference Systems

AUGUST 21, 2025 · BY SHANG ZHU, FEDERICO BIANCHI, WAI TONG CHUNG, ZAIN HASAN, RUPERT WU, CE ZHANG, JAMES ZOU, BEN ATHIWARATKUN

TLDR: Building AI agents to handle complex and long-running engineering tasks requires a different approach than typical AI agent applications. We illustrate key patterns for effective agent development through a real-world case study: using agents to accelerate LLM inference via speculative decoding.



LINKS IN THIS ARTICLE

Introduction

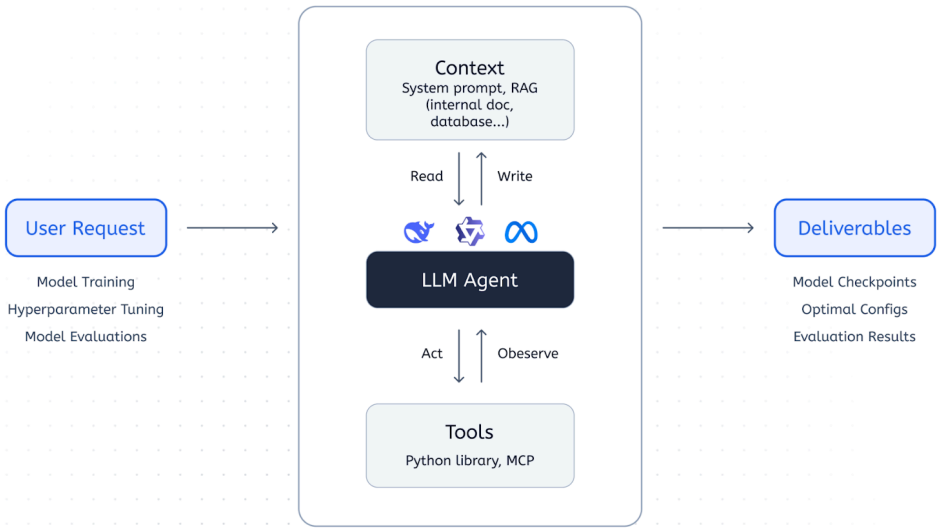
From Cursor and Claude Code to our recently released open data scientist, we've seen the power of coding agents in automating various applications (code understanding, review and debugging, etc.). Much less explored is the end-to-end automation of production workflows using these agents. Most companies have already built different workflows and use-cases for their software, which are often managed by engineering and customer teams who spend a significant amount of time on repetitive infrastructure tasks: configuring environments, launching jobs, monitoring (potentially) long-running processes, collecting results, and orchestrating them all. These workflows are not always easy to automate, owing to complexity or variability in systems design and scale. Furthermore, these workflows could take days (or even weeks) to complete and require constant human oversight to handle frequent failures and edge cases.

At Together AI, we faced this exact challenge while developing our inference optimization workflows. We realized that successfully automating these complex tasks required rethinking how we manage LLM agents.

This blog post distills the key principles we learned from building agents to handle complex engineering tasks, illustrated through our internal pipeline of developing efficient LLM inference algorithms. The agentic system presented here has significantly reduced manual intervention while maintaining consistency and reliability. Our engineers can now oversee the training and control it while the agents take care of the "boring" and repetitive aspects of the work, thus reducing the turnaround time.

AI Agents for Complex Workflow Automation

We found that many coding agents today, such as Claude Code or OpenHands, can effectively follow instructions, edit and execute codebases, and even operate complex workflows. The key design space then becomes the overall architecture in which the agent is embedded. The high level overview of the workflow automation agent is the following:



The key components for agent customization are the context and tools that we equip the agent with, including any internal tools the agent can call and the orchestration of

these various tools in completing a multi-step engineering workflow. Example applications of these architectures might be training or evaluating a model, and optimizing hyperparameters of engineering systems. In the last section of this blog, we present a more detailed case study focusing on training an efficient speculator model to speed up LLM inference.

What Tasks Should We Automate?

To maximize efficiency, tasks selected for automation should meet specific criteria: they need to be:

1. **Verifiable** - with clear success/failure conditions
2. **Well-defined** - having unambiguous steps and boundaries
3. **Supported by existing tools** or tools that can be feasibly integrated

Additionally, they should be generally repetitive for humans, requiring relatively minor adaptations across instances. For example, infrastructure configuration, job monitoring, and hyperparameter tuning in machine learning pipelines often fits this description: they are repetitive yet prone to human error, making them ideal candidates for agentic automation. By focusing on and automating such tasks, teams can offload this routine work to LLM agents while reserving human oversight for high-level decision-making and edge cases.

Now let's dive into what it takes to practically automate these engineering tasks using LLM powered agents!

Six Patterns for Building Automation Agents

Through extensive experimentation, we identified two sets of core patterns that allowed us to build effective autonomous agents for workflow management: Infrastructure Patterns and Behavioral Patterns.

Infrastructure Patterns

Infrastructure patterns center on **how to build your agentic system in practice**; these are useful to define the architecture and environment where the agent is embedded.

Good Tools

The agents we have today are already pretty good for automation. They can understand documentation, execute commands, and react to outputs reasonably well. What these agents rely most on are tools, which can be viewed as a way for the agent to interact with and modify its environment; for instance, `cat` is a tool that allows agents to read files. If the tools we build are well-abstracted and allow the agent to interact with a stable and informative environment (e.g. good error surfacing, nice logging, clean outputs), the agent will be able to complete the task. A stable

environment doesn't mean errors will never occur, but rather that the agent's tools are well-defined and equip it to deal with and recover from these errors.

For example, one of our speculator training scripts was using a custom directory that is only available on our training nodes but was not mounted on the dockerized environment. We initially neglected to consider this and gave only standard instructions to our agent. The agent hit a roadblock with this but was able to find a workaround by specifying new directories for file access. This was because the agent could interact directly with the tools and adapt as needed.

Documentation

Better documentation leads to better outputs, with nearly perfect correlation! Agents operating with comprehensive, well-structured documentation can consistently outperform and execute the right instructions.

Our documentation now includes not just the "happy path" procedures (a step-by-step guide on how a complex engineering task is executed manually), but a few possible failure modes and edge cases. We can also document the *reasoning* behind each step, not just the trace of mechanical actions. This helps agents make better decisions when they encounter novel situations that don't exactly match documented patterns or procedures.

Safe Execution

Working with agents that have significant infrastructure access forced us to reconsider our security model. Unlike human operators who can exercise judgment about risky operations, agents often follow instructions literally and lack intuitive safety mechanisms.

Our solution involves extensive use of development environments that mirror production environments with restricted permissions; to this end we leverage Docker containers that're presumed ephemeral. Agents can experiment freely in these sandboxes using purpose-specific API keys (with fine-grained permissions) without risking production systems. Our agent uses tools that can handle data processing and uploading on our own platform and/or third party providers.

Behavioral Patterns

Behavioral patterns are those that focus on **explaining to the agent how to act**. These are often derived from the documentation.

Manage Parallel Sessions

Traditional agent patterns typically assume short-lived interactions where the agent completes a task (or a short series of simple tasks) and exits. More complex

automation requires agents that can start processes, monitor them over hours or days, and maintain control across interruptions.

Initially, our agents would start long-running processes like model training. They would then either become unresponsive while waiting for lengthy operations to complete, or lose control of critical processes entirely.

Managing parallel workflows can be implemented in several ways. One way is to abstract all tools as simple APIs that the agent can invoke to start services. Another is by instructing the agent to use logging redirection so that standard outputs and errors are always available in specific files for specific processes.

In our implementation, using a multiplexer such as `tmux` solves this in an elegant way. `tmux` sessions persist across agent restarts, allowing recovery from unexpected failures. When an agent encounters an error and needs to restart, it can re-attach to existing sessions and resume monitoring without losing work. Moreover, `tmux` sessions are also available to us; we can login at any time and directly check the status if we want to oversee how the agent is performing a certain task.

Manage Wait Time

Perhaps the most counterintuitive lesson was teaching agents when *not* to act. Early implementations suffered from excessive monitoring overhead: our agents would continuously poll training progress, filling the context windows with repetitive status checks, and wasting computational resources on unnecessary activities.

We resorted to a rather simple pattern: instructing the agent to **estimate** completion time and just sleep and wake up only to check the status of the job after some time. This pattern can also be implemented in other ways: you could have the agent mandatorily respond to user instruction, where you must be aware of various tasks from start to end, or you could implement hooks that notify the agent when a job has been completed. However, `sleep` is, in our experience, the simplest.

Progress Monitoring

Initially, our instructions only described what the agent was to do, without additional context. So if the agent runs a certain command, it might not know that this is a server that takes 10 minutes to spin up. We've since learned that agents require more explicit verification instructions than we initially anticipated.

Early incarnations of our pipeline would execute commands and presume success if no obvious errors occurred. This led to subtle failures that only surfaced minutes later. For instance, the agent would start an inference server and start sending HTTP requests right away without waiting for the inference engine to fully load the model.

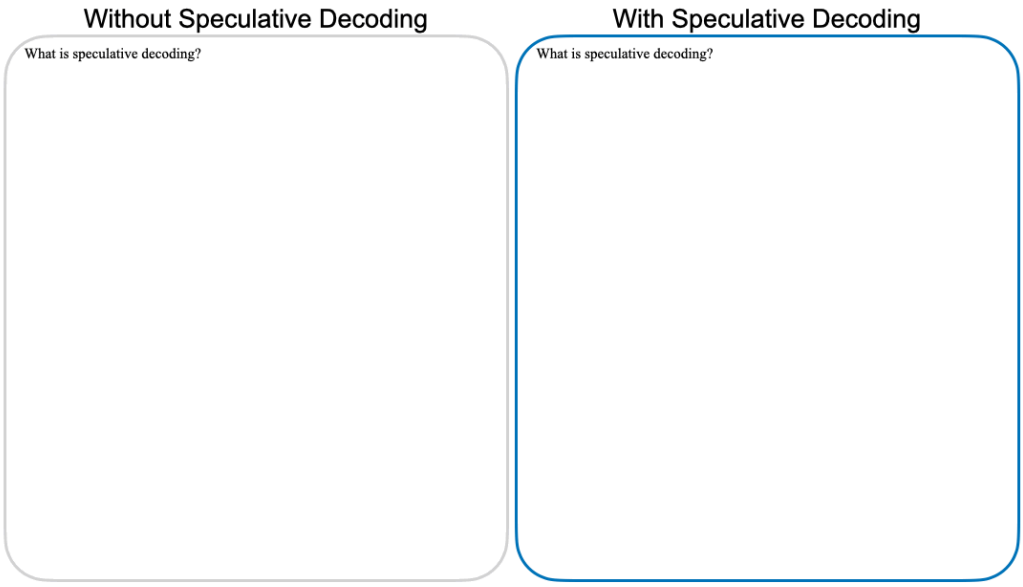
Effective progress monitoring requires improving the documentation and instructing the agent that the commands have some context and might require more time to reach completion: if we explicitly tell the agent that a certain operation is needed to spin up an inference engine for a 32B model, the agent is often good enough to know

that it probably has to wait. In this case, we instructed agents to verify the effective completion of the commands, for example, by checking the status of a server endpoint by running a few test queries.

A Case Study: Automated Speculator Training Pipeline

Speculative decoding is one of the cornerstones of efficient LLM inference, enabling our recent breakthrough inference speed of DeepSeek R1. Conceptually, it's all about using small (draft) models that are trained to speculate what a big (target) model will say before it says it. The big model then only has to verify and correct the small model rather than generate all the tokens from scratch, this can be done for multiple tokens in parallel and thus is a lot faster. It's an awesome technique that allows you to speed up LLM inference 2-3x for very large models!

The animation below shows speculative decoding in action: the slower red tokens are produced by the 670B-parameter DeepSeek-R1 model, while the faster blue/black tokens are from a 8B small LLM trained to mimic R1! If you want to learn more about how speculative decoding works read our blog.



The wide range of other models (open-source or otherwise) for which we can train speculators makes this general process particularly suitable for automation. To demonstrate the principles we discussed in action, we'll walk through how we applied them to automate our speculator training workflow: a complex, multi-day process that previously required frequent human oversight.

The Challenge: Complex Infrastructure Orchestration

Our speculator training pipeline involves multiple interconnected steps: environment setup, data generation from the bigger target model, training the smaller draft model (i.e. speculator) and evaluation. Each step involves intricate command sequences, error handling, and timing considerations. Manual monitoring is both tedious and error-

prone, and a single (perhaps subtle) misconfiguration could invalidate hours or even days of precious compute time.

Architecture and Tools

We built our system using: (1) containerized environments: docker containers optimized for specific tasks, each with pre-configured dependencies and tools, including inference engine, model training and evaluation framework; and (2) agent orchestration: we tested OpenHands and Claude Code in our experiments, both of which are capable of following the complex chain of instructions.

The agents then operate through a sophisticated workflow, following the user-provided documentation:

1. **Task Planning**: Parse experimental requirements and generate an execution plan
2. **Environment Setup**: Select an appropriate environment
3. **Service Management**: Launch and monitor inference engine instances using `tmux` sessions
4. **Pipeline Execution**: Run training workflows with intelligent checkpoint management
5. **Results Aggregation**: Collect and organize all experimental artifacts

Agents-derived Outcome

As previously mentioned, a well-defined and verifiable task is the key to success for automation agents. In this case, the end-to-end speedup is the most critical metric of the speculator training pipeline. Overall, we observed a 1.22x to 1.37x speed up in tokens per second (where the target model is a 32B reasoning LLM) against an internal dataset with variable concurrency, all while minimizing the human intervention of the training pipeline and offloading the repetitive work of configuration and debugging to agents.

Challenges and Future Opportunities

While our system has significantly improved our development workflow efficiency, we acknowledge several persisting challenges: 1) **context management** remains an issue as complex experiments generate extensive logs that strain current context window limits, prompting exploration of better summarization and state management techniques; 2) **novel failure modes** pose difficulties since agents handle documented errors well but struggle with entirely new edge cases, requiring continuous updates to workflow documentation; and 3) **resource optimization** is still evolving, where contemporary agents make conservative allocation decisions, while ongoing efforts strive for more adaptive, real-time resource management.

Meanwhile, the automation of complex workflows using LLM agents opens up vast possibilities, including expanding applications to domains like DevOps, scientific

research, and business process automation. Advances in agent infrastructure—such as better tool integration, persistent session management, and adaptive monitoring—can further enhance reliability and efficiency. Additionally, refining human-LLM interaction through intuitive interfaces and collaborative decision-making could bridge gaps in handling novel edge cases and enable seamless delegation of high-stakes tasks, while retaining human oversight for strategic adjustments.

together.ai